

第一题

```
#include <fcntl.h>
#include <sys/mman.h>
#include <stddef.h>

// fd 必须拥有读写权限以应对 PROT_READ | PROT_WRITE
int fd = open("a.txt", O_RDWR);
size_t ARRAY_LEN = 100 * 1024; // 100K

char *sbuf = (char *)mmap(NULL, ARRAY_LEN, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
char *pbuf = (char *)mmap(NULL, ARRAY_LEN, PROT_READ | PROT_WRITE, MAP_PRIVATE, fd, 0);
```

第二题

总共会产生 50 次 page fault exception，其中 **25** 次会执行 COW。

原因：数组总大小为 100KB，Page 大小为 4KB，因此总共有 $\lceil 100K/4K \rceil = 25$ 个页面。

- 对于 **sbuf**（共享映射），程序首次访问这 25 个页面中的每一个时（按需调页过程），会产生缺页异常以将文件内容读入物理内存，共计产生 **25** 次常规的 page fault。
- 对于 **pbuf**（私有映射），其调页映射初步为指向同一文件的物理页但置为只读。当第一次尝试对其每个页面进行写操作（即 `pbuf[idx] + 1`）时，会触发写保护类型的违规异常，从而由操作系统介入执行 Copy-On-Write (COW)。这 25 个页面共计产生 **25** 次 COW 的 page fault。

第三题

结果如下：

- `sbuf[0] = '1'`
- `sbuf[1] = '1'`
- `pbuf[0] = '2'`
- `pbuf[1] = '1'`

分析：

- 当 `idx = 0` 时：`sbuf[0]` 首先被访问并修改，当前所属物理页的首个字节变为 '1'。紧接着，程序执行 `pbuf[0]` 的写入。这会立刻在同一页上触发 `pbuf` 的 COW 机制。在复制物理页用作私有拷贝的瞬间，原有共享页的第 0 个字节的值已经被改为了 '1'，故 `pbuf` 私有页中的第 0 个字节带着 '1' 被复制了过来。随后，程序继续对它 +1，最终 `pbuf[0]` 变为 '2'。
- 当 `idx = 1` 时：`sbuf[1]` 正常在共享内存递增，由于初始全是 '0'，它会变成 '1'。而对 `pbuf[1]` 来说，由于其所在的第 1 个页面在刚才 `idx = 0` 时已经脱离共享、完成了私有性质的 COW 深拷贝，且拷贝的那一瞬间 `sbuf[1]` 尚未更改（保持为 '0'），故此时 `pbuf[1]` 在其私有页内的原始值为 '0'。执行后变成 '1'。